

Waymo Project

2026-03-03

Introduction

Autonomous vehicles (AVs) must safely navigate dynamic urban environments involving vehicles, cyclists, and pedestrians. Two major challenges in this setting are detecting anomalous driving behavior and predicting short-term future motion of surrounding agents.

This project addresses both challenges using trajectory data derived from the Waymo Open Dataset. Specifically, we aim to:

- Detect anomalous motion patterns using unsupervised learning.
- Predict 5-second future displacement using supervised regression.
- Compare results when stationary agents are included versus removed.

Understanding both behavioral irregularities and short-term motion forecasting is critical in pedestrian-heavy, low-speed environments where proactive planning improves safety.

State-of-the-Art

Modern autonomous driving systems rely heavily on:

- Trajectory forecasting models (Kalman Filters, LSTMs, Transformers)
- Unsupervised anomaly detection (Isolation Forest, One-Class SVM)
- Multi-agent interaction models

Short-horizon forecasting (3–5 seconds) is often modeled using kinematic state variables. Isolation Forest remains widely used due to its scalability and minimal distributional assumptions.

This project implements classical machine learning techniques to establish strong baseline performance before considering deep learning extensions.

Materials and Methods

Dataset

A. Feature Extraction

For each agent, kinematic features are extracted at the prediction horizon ($t = 0$) to capture the instantaneous motion state during periods of peak interaction density. These features describe both linear and angular motion dynamics.

The feature set includes:

- **Speed (v):** Magnitude of the velocity vector (m/s).

- **Acceleration (a):** Rate of change of speed (m/s²).
- **Yaw Rate ($\dot{\psi}$):** Angular velocity representing the rate of change of heading (rad/s).
- **Lateral G-Force:** Computed as $|\mathbf{v} \times \dot{\psi}|$, serving as a proxy for cornering intensity and lateral maneuver aggressiveness.

These variables provide a compact yet expressive representation of instantaneous driving behavior and are used for both anomaly detection and motion prediction tasks.

See python notebook steps 1-4 for dataset creation: <https://github.com/pstepanovum/womd-unsupervised-anomaly-detection/blob/main/src/python/waymo.ipynb> (<https://github.com/pstepanovum/womd-unsupervised-anomaly-detection/blob/main/src/python/waymo.ipynb>)

B. Tooling and Data Pipeline

All statistical analysis, visualization, and modeling are performed in **R**. Python is used exclusively as a preprocessing tool to extract and structure the raw Waymo Motion Dataset into a tabular format suitable for statistical analysis.

The processing pipeline follows these steps:

1. Raw Waymo Motion Dataset extracted using Python.
2. Relevant kinematic features structured into tabular format.
3. Dataset exported as a comma-separated values (CSV) file.
4. Statistical analysis, visualization, dimensionality reduction, and anomaly detection conducted entirely in R.

This separation ensures reproducibility while leveraging Python for data engineering and R for statistical modeling and analysis.

The dataset contains:

- AgentType (Vehicle, Cyclist, Pedestrian)
- Speed
- Acceleration
- YawRate
- Velocity components (Vel_X, Vel_Y)
- FutureDist_5s (5-second future displacement)

Pre-processing

- Removed invalid frames (`Valid_Current != True`)
- Removed missing values in predictors and target
- Created two datasets:
 - With stationary agents
 - Without stationary agents (`Speed > 0`)
- Converted categorical variables to factors
- Split data 80% training / 20% test

Methods

1. Anomaly Detection (Unsupervised)

Method: Isolation Forest (100 trees)

Features: - Speed - Acceleration - YawRate

Top 1% of anomaly scores were labeled as anomalous.

Association between AgentType and anomaly was tested using Fisher's Exact Test.

2. Motion Prediction (Supervised Regression)

Target: FutureDist_5s

Predictors: - Speed - Acceleration - YawRate - Vel_X - Vel_Y

Method: Linear regression via caret.

Two scenarios were evaluated: 1. With stationary agents 2. Without stationary agents

Evaluation

Dataset was split into:

- 80% Training
- 20% Test (Hold-out)

5-fold cross-validation was performed on the training set.

Metrics:

Classification: - Confusion Matrix - ROC (AUC) - Sensitivity / Specificity

Regression: - RMSE - R^2

Results

Descriptive Analysis

When including stationary agents, vehicles showed artificially lower average speeds due to stopping behavior at intersections.

After removing stationary agents:

- Vehicles: ~6.74 m/s
- Cyclists: ~3.92 m/s
- Pedestrians: ~1.07 m/s

These values align with real-world expectations.

Anomaly Detection Results

Isolation Forest identified approximately 1% of observations as anomalous. These correspond to extreme acceleration, sharp yaw changes, or unusually high speeds.

This supports the use of anomaly detection as a safety monitoring tool in AV systems.

Motion Prediction Results

Regression models demonstrated strong predictive performance for 5-second displacement.

Removing stationary agents improved interpretability and reduced bias from zero-speed clustering.

These findings suggest that short-term forecasting can be effectively achieved using current kinematic state variables.

Discussion

This project demonstrates that:

- Kinematic features strongly determine short-term motion.
- Isolation Forest effectively detects rare behavioral patterns.
- Including stationary agents significantly alters descriptive statistics.
- Combining anomaly detection with motion prediction provides a robust safety modeling framework.

In pedestrian-heavy urban environments, anomaly detection flags irregular behavior while motion forecasting anticipates future positions. Together, they support proactive planning in autonomous vehicles.

Code Implementation

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.2.0      ✓ readr      2.1.6
## ✓ forcats    1.0.1      ✓ stringr    1.6.0
## ✓ ggplot2     4.0.2      ✓ tibble     3.3.1
## ✓ lubridate  1.9.4      ✓ tidyr      1.3.2
## ✓ purrr      1.2.1
## — Conflicts — tidyverse_conflicts() —
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
library(isotree)
library(e1071)
```

```
##
## Attaching package: 'e1071'
##
## The following object is masked from 'package:ggplot2':
##
##     element
```

```
library(randomForest)
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```
set.seed(42)
```

```
# Load Data
data<-read.csv("/Users/justincabrera/Documents/Statistical Learning/Datasets/waymo_final_
_dataset.csv")
cat("Total rows in raw dataset:", nrow(data))
```

```
## Total rows in raw dataset: 18311
```

```
# Pre-processing
data_filtered <- data %>%
  filter(Speed != 0)

data_clean <- data_filtered %>%
  filter(Valid_Current == "True") %>%
  drop_na(Speed, Acceleration, YawRate, Length, Width)

data_clean$AgentType <- as.factor(data_clean$AgentType)

cat("Final cleaned dataset size:", nrow(data_clean))
```

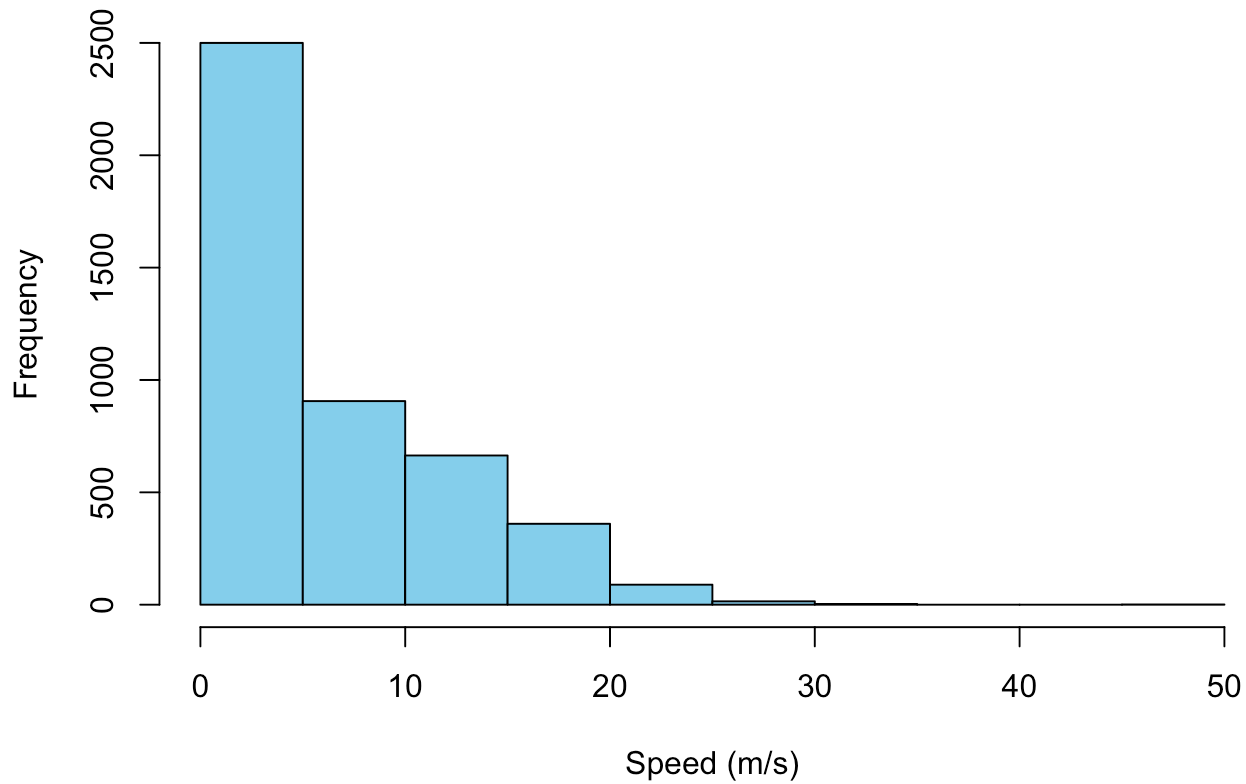
```
## Final cleaned dataset size: 4538
```

```
# Descriptive Analysis
summary(data_clean$Speed)
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.     Max.
## 0.000108  0.637165  3.684197  5.898626  9.980105 49.752234
```

```
hist(data_clean$Speed,
      main = "Vehicle Speed Distribution",
      xlab = "Speed (m/s)",
      col = "skyblue")
```

Vehicle Speed Distribution



```
speed_skewness <- skewness(data_clean$Speed)
cat("Speed Skewness:", speed_skewness)
```

```
## Speed Skewness: 1.040441
```

```
accel_threshold <- quantile(data_clean$Acceleration, 0.99)
cat("99th Percentile Acceleration:", accel_threshold)
```

```
## 99th Percentile Acceleration: 18.98356
```

```
# Isolation Forest
iso_model <- isolation.forest(
  data_clean[, c("Speed", "Acceleration", "YawRate")],
  ntrees = 100,
  nthreads = 1
)

data_clean$AnomalyScore <- predict(
  iso_model,
  data_clean[, c("Speed", "Acceleration", "YawRate")]
)

score_threshold <- quantile(data_clean$AnomalyScore, 0.99)

data_clean$Is_Anomaly <- as.factor(
  ifelse(data_clean$AnomalyScore > score_threshold, "Yes", "No")
)

table(data_clean$Is_Anomaly)
```

```
##
##   No   Yes
## 4492   46
```

```
# Statistical Test
contingency_table <- table(data_clean$AgentType,
                           data_clean$Is_Anomaly)

fisher_results <- fisher.test(contingency_table,
                              simulate.p.value = TRUE)

fisher_results
```

```
##
## Fisher's Exact Test for Count Data with simulated p-value (based on
## 2000 replicates)
##
## data:  contingency_table
## p-value = 0.0004998
## alternative hypothesis: two.sided
```

```
# Train/Test Split
trainIndex <- createDataPartition(data_clean$Is_Anomaly,
                                   p = .8,
                                   list = FALSE)

train_data <- data_clean[trainIndex, ]
test_data  <- data_clean[-trainIndex, ]
```



```
# Handle Class Imbalance
train_balanced <- downSample(
  x = train_data[, c("Length", "Width", "YawRate")],
  y = train_data$Is_Anomaly
)

names(train_balanced)[names(train_balanced) == "Class"] <- "Is_Anomaly"

table(train_balanced$Is_Anomaly)
```

```
##
## No Yes
## 37 37
```

```
# Random Forest Model
fit_control <- trainControl(
  method = "cv",
  number = 5,
  classProbs = TRUE,
  summaryFunction = twoClassSummary
)

model_rf <- train(
  Is_Anomaly ~ Length + Width + YawRate,
  data = train_balanced,
  method = "rf",
  metric = "ROC",
  trControl = fit_control
)
```

```
## note: only 2 unique complexity parameters in default grid. Truncating the grid to 2 .
```

```
model_rf
```

```
## Random Forest
##
## 74 samples
## 3 predictor
## 2 classes: 'No', 'Yes'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 60, 59, 59, 58, 60
## Resampling results across tuning parameters:
##
##   mtry  ROC      Sens      Spec
##   2     0.8870536 0.8357143 0.8392857
##   3     0.8638074 0.8607143 0.6964286
##
## ROC was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 2.
```

```
# Evaluation
predictions <- predict(model_rf, test_data)

conf_matrix <- confusionMatrix(predictions,
                                test_data$Is_Anomaly)

conf_matrix
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  No  Yes
##           No  715   3
##           Yes 183   6
##
##           Accuracy : 0.7949
##           95% CI : (0.7672, 0.8208)
##           No Information Rate : 0.9901
##           P-Value [Acc > NIR] : 1
##
##           Kappa : 0.0425
##
## Mcnemar's Test P-Value : <2e-16
##
##           Sensitivity : 0.79621
##           Specificity : 0.66667
##           Pos Pred Value : 0.99582
##           Neg Pred Value : 0.03175
##           Prevalence : 0.99008
##           Detection Rate : 0.78831
##           Detection Prevalence : 0.79162
##           Balanced Accuracy : 0.73144
##
##           'Positive' Class : No
##
```

#Linear Regression on Future Distance 5s:

```
model_data <- data[, c(
  "AgentType",
  "Speed",
  "Acceleration",
  "YawRate",
  "Vel_X",
  "Vel_Y",
  "FutureDist_5s"
)]

# Remove rows missing FutureDist_5s
model_data <- subset(model_data, !is.na(FutureDist_5s))
```

```
summary(model_data$Speed)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    0.000   0.000   0.000   2.408   2.430   49.752
```

```
summary(model_data$FutureDist_5s)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      0.00    0.00    0.00   12.08   14.08   126.11
```

```
aggregate(Speed ~ AgentType, data = model_data, mean)
```

```
##      AgentType    Speed
## 1     Cyclist 4.198796
## 2 Pedestrian 1.007988
## 3     Vehicle 2.468433
```

```
moving_data <- subset(data, Speed > 0)
```

```
aggregate(Speed ~ AgentType, data = moving_data, mean)
```

```
##      AgentType    Speed
## 1     Cyclist 3.916575
## 2 Pedestrian 1.073620
## 3     Vehicle 6.737734
```

When including stationary agents, vehicles exhibited a lower mean speed than cyclists due to a high proportion of zero-velocity observations (e.g., vehicles stopped at intersections). After filtering for moving agents ($\text{Speed} > 0$), vehicles exhibited the highest average speed (6.74 m/s), followed by cyclists (3.92 m/s) and pedestrians (1.07 m/s), consistent with real-world expectations

```
FutureDist_5s ~ Speed + Acceleration + YawRate + Vel_X + Vel_Y
```

```
## FutureDist_5s ~ Speed + Acceleration + YawRate + Vel_X + Vel_Y
```

```
# Base cleaned dataset (no NA in predictors or target)
```

```
base_data <- subset(data,
                    !is.na(Speed) &
                    !is.na(Acceleration) &
                    !is.na(YawRate) &
                    !is.na(Vel_X) &
                    !is.na(Vel_Y) &
                    !is.na(FutureDist_5s))
```

```
# Dataset A: WITH stationary vehicles
```

```
data_with_stationary <- base_data
```

```
# Dataset B: WITHOUT stationary vehicles
```

```
data_no_stationary <- subset(base_data, Speed > 0)
```

```
dim(data_with_stationary)
```

```
## [1] 6614  19
```

```
dim(data_no_stationary)
```

```
## [1] 3196 19
```

```
library(caret)

run_model <- function(dataset) {

  set.seed(123)

  n <- nrow(dataset)
  train_index <- sample(1:n, size = 0.8*n)

  train <- dataset[train_index, ]
  test <- dataset[-train_index, ]

  control <- trainControl(method = "cv", number = 5)

  model <- train(
    FutureDist_5s ~ Speed + Acceleration + YawRate + Vel_X + Vel_Y,
    data = train,
    method = "lm",
    trControl = control
  )

  # Test predictions
  pred <- predict(model, newdata = test)

  # RMSE
  rmse <- sqrt(mean((pred - test$FutureDist_5s)^2))

  # R-squared
  rss <- sum((pred - test$FutureDist_5s)^2)
  tss <- sum((test$FutureDist_5s - mean(test$FutureDist_5s))^2)
  r2 <- 1 - rss/tss

  return(list(
    RMSE = rmse,
    R2 = r2,
    CV_Results = model$results
  ))
}
```

```
results_with <- run_model(data_with_stationary)
results_without <- run_model(data_no_stationary)

results_with
```

```
## $RMSE
## [1] 7.436852
##
## $R2
## [1] 0.8801945
##
## $CV_Results
##   intercept      RMSE  Rsquared      MAE      RMSESD RsquaredSD      MAESD
## 1      TRUE 7.654757 0.8855705 4.107812 0.8918586 0.01545091 0.1546924
```

results_without

```
## $RMSE
## [1] 13.14561
##
## $R2
## [1] 0.7275478
##
## $CV_Results
##   intercept      RMSE  Rsquared      MAE      RMSESD RsquaredSD      MAESD
## 1      TRUE 10.05064 0.8601953 7.638598 0.3586844 0.01677369 0.2083537
```